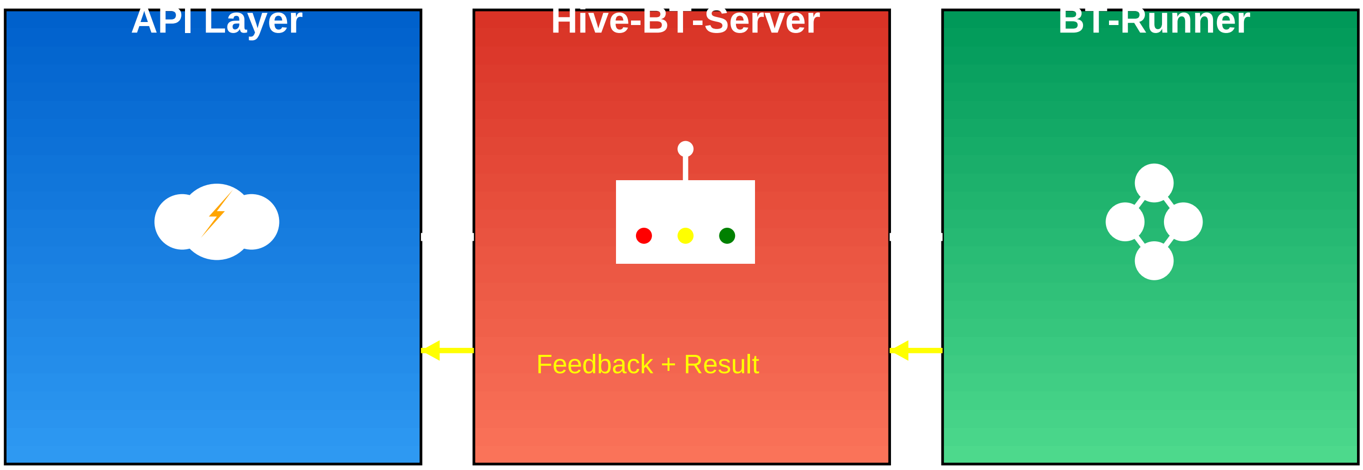
Hive BT System

Modern 3-Layer Architecture for Multi-Behavior Robotics

API Layer • Hive-BT-Server • BT-Runner • Groot2 • ROS 2 Actions


API Layer
Hive-BT-Server
BT-Runner
Feedback + Result

API Layer – HTTP → ROS 2 Action

Client sends HTTP

POST /tasks { id: 1|2 }

Validate & Build Goal

Create ExecuteBehavior.Goal

Send to Hive

Action goal -> /hive/execute_behavior

Respond

202 Accepted + task_id

Hive-BT-Server – Router + Monitor

Route

id → behavior (1→SimpleTurtle, 2→GoToA)

↓

Forward to Runner

Action goal -> /bt_runner/execute_behavior

↓

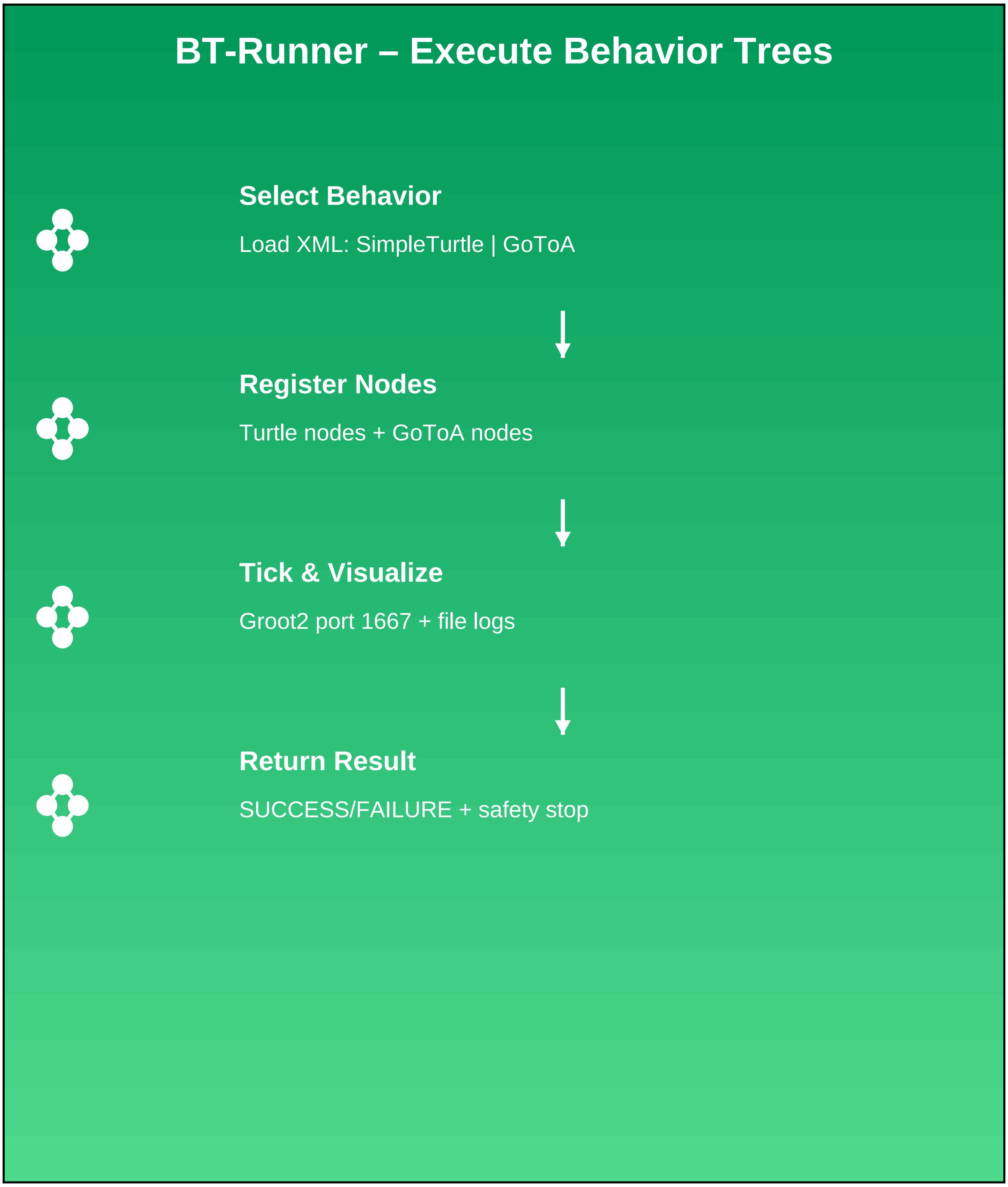
Relay Feedback

Publish /hive/status JSON

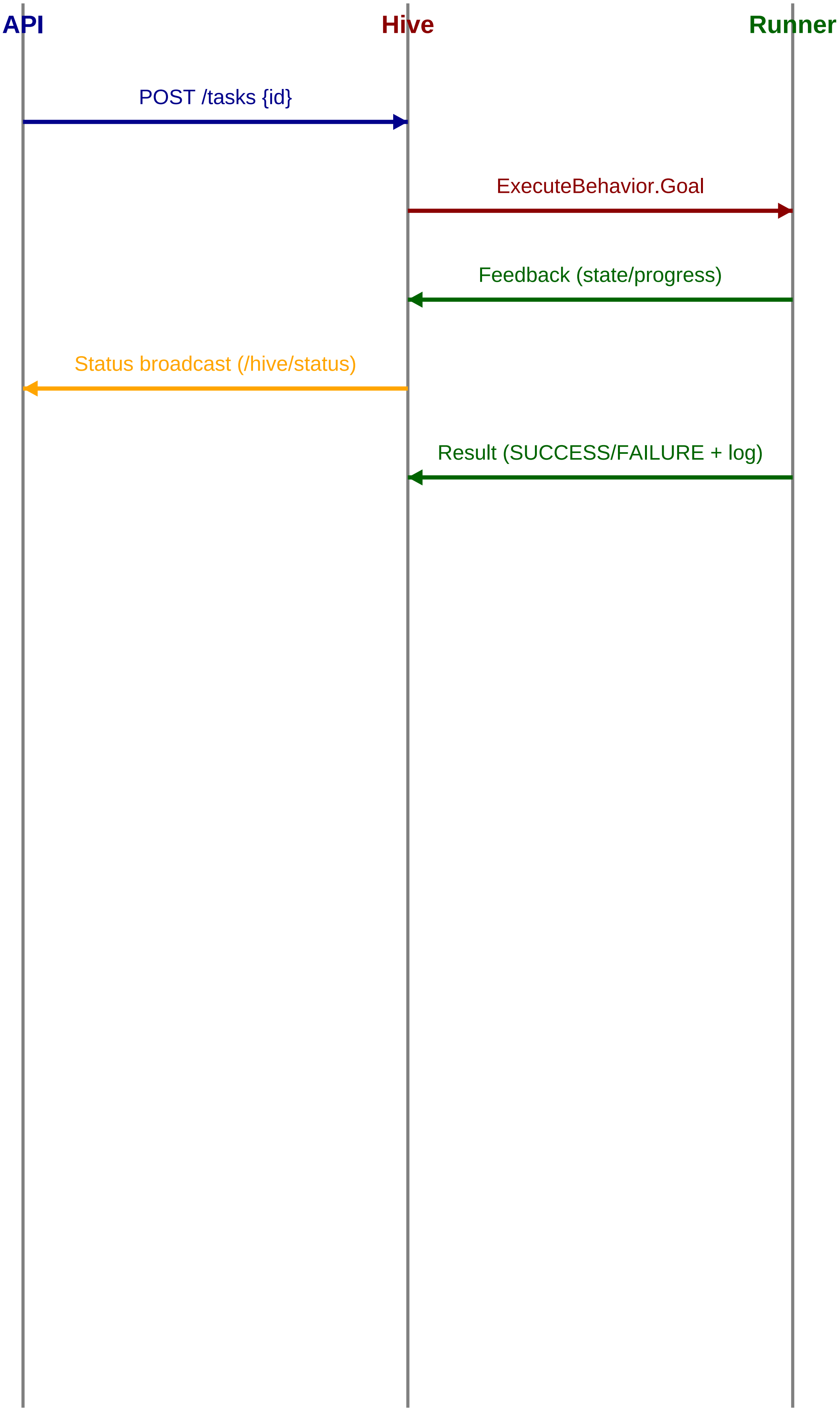
↓

Finalize

Return SUCCESS/FAILURE + logs


BT-Runner – Execute Behavior Trees
Select Behavior
Load XML: SimpleTurtle | GoToA
Register Nodes
Turtle nodes + GoToA nodes
Tick & Visualize
Groot2 port 1667 + file logs
Return Result
SUCCESS/FAILURE + safety stop

End-to-End Sequence



Run, Debug & Scale

• Run three terminals: Runner, Hive, API (see README commands).
• Trigger with curl: **id=1** (SimpleTurtle), **id=2** (GoToA).
• Visualize in Groot2 at port 1667; slow tick_ms for clarity.
• Track via **/hive/status** and per-task **.btlog** files.
• Add behaviors by registering new nodes + XML; map new IDs in runner.
• Scale to multi-robot by namespacing runner/action names.